



NPUFort: A Secure Architecture of DNN Accelerator Against Model Inversion Attack

Xingbin Wang
State Key Laboratory of Information
Security, Institute of Information
Engineering, CAS
University of Chinese Academy of
Sciences
Beijing, China
wangxingbin@iie.ac.cn

Rui Hou*
State Key Laboratory of Information
Security, Institute of Information
Engineering, CAS
Beijing, China
hourui@iie.ac.cn

Yifan Zhu
State Key Laboratory of Information
Security, Institute of Information
Engineering, CAS
University of Chinese Academy of
Sciences
Beijing, China
zhuyifan@iie.ac.cn

Jun Zhang
Hubei University of Arts and Science
Beijing, China
zhangjun@hbuas.edu.cn

Dan Meng
State Key Laboratory of Information
Security, Institute of Information
Engineering, CAS
Beijing, China
mengdan@iie.ac.cn

ABSTRACT

Deep neural network (DNN) models are widely used for inference in many application scenarios. DNN accelerators are not designed with security in mind, but for higher performance and lower energy consumption. Hence, they are suffering from the security risk of being attacked. The insecure design flaws of existing DNN accelerators can be exploited to recover the structure of DNN model from the plain instructions, thus the runtime environment can be controlled to obtain the weights of DNN model. Furthermore, the structure of DNN model running on the accelerator is acquired by the side channel information and interrupt status register. To protect general DNN accelerator from being attacked by model inversion attack, this paper proposes a secure and general architecture called NPUFort, which guarantees the confidentiality of the parameters of DNN model and mitigates side-channel information leakage. The experimental results demonstrate the feasibility and effectiveness of the secure architecture of DNN accelerators with negligible performance overhead.

CCS CONCEPTS

• **Security and privacy** → **Hardware attacks and countermeasures.**

KEYWORDS

DNN accelerator, Deep neural networks, Secure architecture, Model inversion

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

CF '19, April 30-May 2, 2019, Alghero, Italy

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6685-4/19/05...\$15.00

<https://doi.org/10.1145/3310273.3323070>

1 INTRODUCTION

Deep neural networks (DNNs) have been thriving in many application areas such as image and speech recognition. Training deep neural networks require a large amount of training data and powerful computing resources. Furthermore, training data such as medical images and financial data are generally private and sensitive. All of these factors make the well-trained deep learning models become confidential and commercial.

The energy-efficient DNN accelerators possess promising versatility and performance. Trained a prediction models are deployed on the DNN accelerators at edge to process sensor data. With the proliferation of DNN accelerators, accompanying attacks happen. Mobile and embedded systems are physically exposed to potential attackers who could steal and duplicate the model [30]. When the trained model is running on DNN accelerator with low latency, an attacker could steal the DNN model through side-channel attacks [11, 30]. Hua et al. [11] design an attack to steal DNN models running on DNN accelerators, and infer weights of DNN model through some specific operations such as zero pruning [9]. Papernot et al. [21] and Tramèr et al. [31] exploit the information leakage from APIs to obtain the weights of DNN models from the machine-learning-as-a-service (MLaaS) cloud platforms. As shown in Table 1, a variety of attack methods and different platforms have been reported.

The well-trained DNN models such as medical image recognition models are unable or unwilling to be stolen for invading intellectual property or using stolen model to launch potential model-oriented attacks. To avoid these attacks, secure countermeasures should be applied to the DNN accelerators for protecting DNN models. Unfortunately, the design of existing DNN accelerators [1, 6, 8, 12, 22] has not taken into account the security of DNN model. They are suffering from the memory and timing side channel attack as listed

*Corresponding author: Rui Hou (hourui@iie.ac.cn).

in Table 1. Furthermore, the host processor could be hijacked to read the instruction registers and local DRAM from DNN accelerator.

To deal with the above problems, we propose a secure architecture for DNN accelerator called *NPUFort* which is high energy efficiency. *NPUFort* is an extension of general DNN accelerators with a security module designed from instruction flow, data flow, and control flow. Although the performance optimization of DNN accelerators is also closely combined with additional security issues, it is a trade-off problem. Compared with the original architecture without secure countermeasures, *NPUFort* alleviates the problem and improves the performance by approximately 1.112 $\times$ while guaranteeing the security with negligible performance overhead.

Table 1: The attack methods and platforms of DNN models.

Attack methods	Platforms	Literatures
Synthetic samples	MLaaS	Tople et al.[30]
Learn by learner	MLaaS	Juuti et al.[21]
Queries model via APIs	MLaaS	Wang et al.[31] [32]
Power side channel	ARM Cortex-M3	Batina et al.[2]
Cache side channel	CPU	Yan et al.[34]
GPU side channel attack	GPU	Pessl et al.[16]
Memory side channel	Accelerator	Hua et al.[11]

The main technical contributions of the paper are as follows:

- This paper comprehensively analyzes the insecure design flaws of existing DNN accelerators. The runtime environment can be controlled to query instruction registers for making duplicated instruction file, and obtain the weights of DNN model.
- The paper firstly proposes a secure architecture of DNN accelerator called *NPUFort*, which extends the general DNN accelerators with an instruction security unit, a data security unit. The critical feature maps are firstly applied to safeguard the security of DNN model running on DNN accelerator.
- We implement the secure architecture based on NVDLA [17] at the Register Transfer Level (RTL), and evaluate the feasibility and effectiveness of the secure architecture. Then we also present the quantitative relationship between secure architecture design and performance overhead.

The rest of this paper is organized as follows. In section 2, we illustrate the general DNN accelerator and model inversion attack on DNN accelerator. In section 3 presents the threat model. In section 4, we show the model inversion attack based on instruction file. Section 5 gives the model inversion attack based on side channel information leakage. Section 6 proposes the secure architecture of DNN accelerator. Section 7 describes the experimental platform and detailed analysis of the secure architecture. Section 8 shows the related work. Section 9 concludes the paper.

2 BACKGROUND

2.1 Understanding Deep Learning Accelerator

The trained model is generally deployed on DNN accelerators to make real-time predictions on new inputs. In order to provide a simple, flexible, and robust edge deep learning solution, the architecture which facilitates the standardization of DNN accelerators designed to be both scalable and highly configurable. And existing DNN accelerators are equipped with dedicated instruction sets and

compilers (include runtime environment). For example, Diannao [15] proposed a domain-specific Instruction Set Architecture (ISA) for DNN accelerators called Cambricon [15]. The instruction set contains a load-store architecture that integrates scalar, vector, matrix, logical, data transfer, and control instructions. The compiler is designed based on the Cambricon instruction set. NVIDIA has opened up a DNN accelerator called NVDLA [17, 19], which contains register instructions for each function module and the corresponding compiler. The Versatile Tensor Accelerator (VTA) [5] is an open, generic, and customizable DNN accelerator with a complete TVM-based compiler stack. DNN accelerators based on instruction sets and compilers have become dominant and increasingly popular with academia and industry.

The general architecture of the DNN accelerator is showed in Figure 1. It consists of two parts: software stack and hardware stack. The software stack contains compiler and runtime environment, and the hardware stack has the DNN accelerator and off-chip memory (DRAM). The architecture of DNN accelerator based on the instruction set can be broken down into three main components: compiler, arrays of processing elements (PEs), control processor. The compiler interprets the parsed network and converts it into a set of hardware layers, then maps each network operation onto the functional block of DNN accelerator. Compiler generates instruction flow for function operations of DNN accelerator to execute inference. The PEs are responsible for carrying out the computation of DNN layers. Control processor receives instructions and controls instruction execution. The PE arrays and control processor are connected via a network-on-chip.

DNN accelerators and CPUs typically interact with data through shared memory or data transfer interface such as PCIe interface or other SoC bus interface. CPU generally receives instructions generated from the compiler and sends them to the DNN accelerator for execution. To complete a layer of the network, the DNN accelerator with general architecture derived from [4, 5] has to receive instructions from the host CPU. Then instruction flow generated by compiler controls the accelerator to inference a DNN layer-by-layer based on natural structure of the network. At the beginning of the DNN layer processing, the input feature maps and weights are stored in the off-chip memory. During the processing, the data is transported to PE arrays for performing matrix multiplications and accumulations. After processing the DNN layer, the computed output feature maps after activation and pooling are written back into the off-chip memory. After completing the layer, the accelerator generates an interrupt to inform the CPU and let it send the instructions of next layer. When the accelerator finished the computation of all layers in a forward-propagation fashion, it generates the probability for each class as the result of inference phase.

Based on the general DNN accelerators, we have obtained the following common features.

- General DNN accelerators have their own instruction sets and corresponding compilers.
- General DNN accelerators have similar processing flow for the network inference based on the instructions.
- General DNN accelerators have a similar microarchitecture.

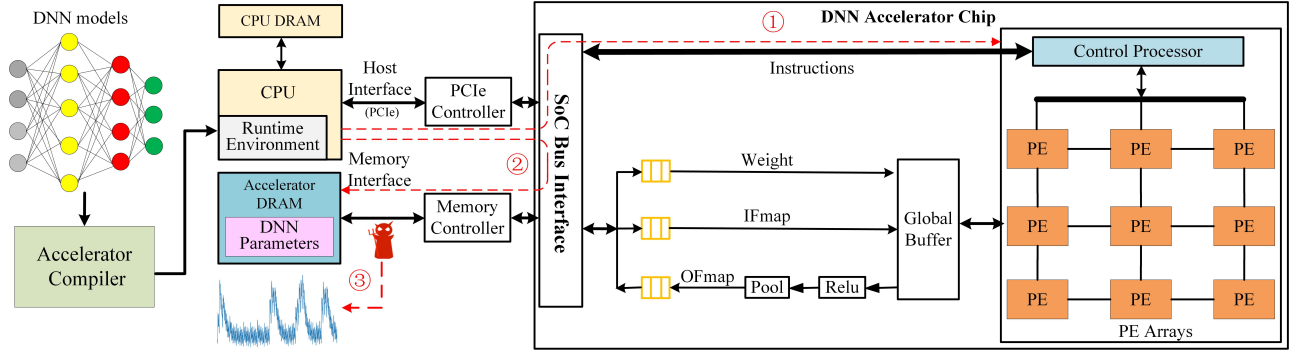


Figure 1: General DNN accelerator with three attack paths

- General DNN accelerators need to interact with the CPU through commodity system-level or on-chip level interconnect (e.g, PCIe or AXI), including data interaction and instruction interaction.

2.2 Model Inversion Attack on DNN accelerator

If an attacker has an opportunity to get the plain instruction file, he may launch model inversion attack to obtain the structure of DNN from the instructions and the address where the weights are stored. The attacker can also know the correspondence between weight data and the layer by analyzing the currently executed layer and reading weight data. Even without instruction file, the attacker who has the ability to attack the runtime environment can query the configuration register or insert instructions to constitute an instruction file ①. Then the weights of DNN model can be read by the runtime environment which is hijacked by the attacker ②.

As the feature maps and weights are too large, they are usually stored in off-chip memory of DNN accelerator. An attacker can reveal memory access patterns in which each access is a read or write operation [11], and can also read interrupt status register to know the current interrupt corresponding to the completed function module ③. Based on the leverage information, the attacker can further infer the features of the network structure such as the number of layers, the size of filters, layers dependencies. In addition, the execution time of each function module also indicates the amount of computation. The possible network structures can be inferred from these information. Then a few potential structures of DNN model are candidates.

3 THREAT MODEL

In this study, we primarily analyze the attack surfaces of general DNN accelerators and how to mitigate them. In order to obtain the structure and weights of the model, an attacker needs to rely on the standard assumptions which can be divided into two categories corresponding to two attack scenarios.

- Assuming that the host processor is insecure and vulnerable to attack. The host processor controlled by the attacker can operate the runtime environment to read instruction register from DNN accelerator. The attacker can obtain the instruction file from host processor, and control host processor to read weights from DNN accelerator.

- The compromised OS controlled by an attacker can read the interrupt status of DNN accelerator. An attacker can observe memory accesses by physical probing of memory bus or side channel information leakage by page faults and cache conflicts [33] or repeated reading memory to detect changes. In addition, an attacker can exploit DRAM addressing for cross-CPU attack to obtain the information of memory access [23]. To the best of our knowledge, there are no DNN accelerators that deploy side-channel countermeasures. So these assumptions are realistic.

Two typical types of attack scenarios in existing DNN accelerators are realistic in many important application. The goal of the first attack scenario is to obtain the structure and weights of the target model running on DNN accelerators directly. The second attack scenario is to steal the structures of target model based on side-channel information leakage. Then the attacker attempts to construct a duplicated DNN model that has a comparable classification accuracy to the original target model [11].

4 MODEL INVERSION ATTACK BASED ON INSTRUCTION FILE

In this section, we propose an effective attack to achieve the goal of getting the structure and weights simultaneously. According to the correspondence between the instruction and the various operations of DNN model, an attacker firstly analyzes input instruction file to obtain network's architecture and the address of weights. Then host CPU reads the data from the weight's address and takes them away. In order to carry out such an attack, we apply NVDLA as examples to illustrate.

Study case-1: NVDLA. NVDLA is built as a series of pipeline stages, they are used to handle each layer in whole or in part. The main operations contain a convolution operation, an activation operation which is a single data processor, and a pooling operation which is a planar data processor. Convolution operations include convolution DMA, convolution buffer, convolution sequence controller, convolution MAC array and convolution accumulator [20]. Figure 2 shows the results of instructions interpretation for NVDLA. The height and weight of kernel, stride and pad from the convolution instructions [17] can be seen. In this way, activation function and pooling are indicated by their instructions, respectively. In

Name	Register address	
D_DAI_ADDR_LOW_0	0x5034 //input data address	
D_WEIGHT_ADDR_LOW	0x507c // weight address	
D_BN_BASE_ADDR_LOW	0xa044 //address of the bias	
D_CONV_STRIDE	0x50b0 //convolution x stride and y stride	Kernel size, Stride size, Pad size Convolution
D_ZERO_PADDING	0x50b4 p//adding size	
D_ZERO_PADDING_VALUE	0x50b8 //padding value	
D_DATAIN_SIZE_I	0x5020 //input cube' s channel	
D_WEIGHT_BYTES	0x5080 //total bytes of weight	Activation Function
D_WEIGHT_SIZE_I	0x5070 //Number of kernels	
D_DP_BS_CFG	0xb058 // BS module of activation function	
D_DP_BN_CFG	0xb06c // BN module of activation function	Pooling
D_DP_EW_CFG	0xb080 //EW module of activation function	
D_POOLING_KERNEL_CFG	0xc038 //Kernel width and kernel stride	
D_POOLING_PADDING_CFG	0xd040 // padding size	
INTR_STATUS	0x000c //Interrupt status	
//Interrupt status values 0x00010000 CDMA_DAT_DONE_STATUS		Read feature maps done
//Interrupt status values 0x00040000 CDMA_WT_DONE_STATUS		Read weights done
//Interrupt status values 0x00100000 CACC_DONE_STATUS		Convolution done
//Interrupt status values 0x00000001 SDP_DONE_STATUS0		Activation function done

Figure 2: Analysis of NVDLA Instructions

addition, every concrete operation is completed from the interrupt status such as complete convolution operation or complete activation function.

5 THE SECURE ARCHITECTURE OF DNN ACCELERATOR

In order to reduce attack surfaces of DNN accelerator, we propose a secure architecture called *NPUFort* to prevent attackers from obtaining plaintext instructions and weights. *NPUFort* reduces memory and timing side-channel information leakage of DNN model. Figure 3 shows the secure architecture of DNN accelerator. *NPUFort* is an extension of general DNN accelerator with an instruction security unit, a data security unit. Firstly, the read of configurable registers including interrupt registers by host CPU should be disabled. Furthermore, in order to prevent attacker from inserting instructions, *bypass and link check* should be applied to verify instructions. Secondly, tiny AES which uses the CTR mode (AES-CTR) should be exploited to encrypt and decrypt the data running on DNN accelerator. Certainly, *critical feature maps* are chosen to encrypt and decrypt for better performance and security. The clocks that these security units operate are synchronized with the clock of DNN accelerator.

5.1 Instruction and Data Security Unit

For the security risks of general DNN accelerators, a natural idea is to encrypt instructions and weights, while simple encryption suffers from many problems and challenges. The security and performance of encryption is a trade-off. How to select the proper encryption algorithm and how the key is stored and transmitted?

To address these issues, we investigated many encryption algorithms and selected AES-CTR as an encryption algorithm on the premise of security and performance. In order to improve the efficiency of the encryption algorithm, AES-CTR is designed as a pipeline to execute encryption and decryption in parallel. On the

pipeline, the cycle latency of AES-CTR implementation is 27. As the security of the AES encryption algorithm is guaranteed by both the key and the counter, the initial value of the counter and the key for each layer is generated by the Linear Feedback Shift Register (LFSR).

However, the key and the initial value of the counter which are used for instruction and weight decryption need to be passed from the CPU. To transfer the key, we need to choose an asymmetric encryption algorithm called RSA (Rivest-Shamir-Adleman) for key transmission. The DNN accelerator obtains the key by RSA algorithm, and the key is used for decrypting instructions and weights. To prevent an attacker from inserting or modifying instructions, we need to verify the decrypted instructions. We propose an instruction verification method called *bypass and link check*. The *bypass and link check* mainly consists of cyclic redundancy check algorithm. Since the instruction check is a bypass check, when the check error occurs, it blocks the DNN accelerator from writing instructions and informs the CPU to re-send. That is why it does not affect the performance of the accelerator.

5.2 Reduce the Timing Side Channel by Critical Feature Maps

To prevent attacker from using intermediate data [26, 29], feature maps need to be encrypted. But encrypting all feature maps will result in increasing energy consumption. In addition, the use of symmetric encryption algorithms also causes RAW information leakage. It is unnecessary to encrypt intermediate data, and not all feature maps are important. So we just need to encrypt the important feature maps and propose a novel encryption method based on the *critical feature maps*. Compared with encrypting all feature maps, this approach reduces power consumption and obscures the boundary between layers to enhance security. Due to the read-write latency between the encrypted feature map and the unencrypted feature map, it is impossible for an attacker to obtain the accurate execution time of each layer through timing side channel information leakage.

Li et al. [13] measured the importance of each feature map by calculating its absolute weight sum. Hu et al. [10] think that the average percentage of zeros of each filter is an importance score. In addition, Yang et al. [35] use energy estimation based on their energy consumption to determine the importance of the feature map. We chose the above three points as important scoring criteria.

Due to the pipelined encryption algorithm, the latency of the encrypted critical feature map is related to the number and correlation of the critical feature maps. The more the number of encrypted critical paths are, the greater the performance loss is. In order to make the encryption of critical feature map safer and the performance loss less, the channel density and channel correlation coefficient for the layer are introduced. The channel density is defined by the formula: $\rho = \frac{F_{key}}{F_{sum}}$. F_{key} is the number of critical feature maps, F_{sum} is the number of all the feature maps for the layer. The channel correlation coefficient is defined by the equation: $K = \frac{F_c}{F_{sum}}$. F_c is the number of continuous critical feature maps. K_g is the number of the continuous critical feature maps group which is proportional to the performance loss. When the channel density is close to 0.5, data

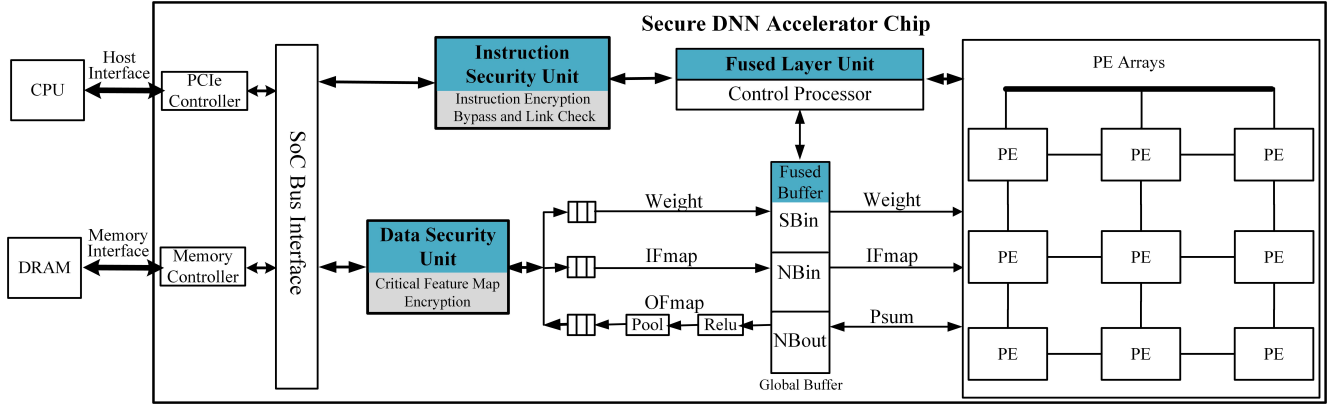


Figure 3: The secure architecture of DNN accelerator

security unit has higher security. When the channel correlation coefficient is close to 0, it has the higher security.

In order to implement data security unit for protecting intermediate feature maps, important feature maps should be selected first based on the three criterions. Then the security of them could be evaluated with channel density and correlation coefficient. Certainly, to achieve a balance of security and performance, we will manually add or remove nodes for critical feature maps. Last but not least, DNN accelerator applies the encryption algorithm to encrypt and decrypt critical feature maps.

6 EVALUATION

6.1 Methodology

Secure architecture based on NVDLA (SecNVDLA). In order to accurately evaluate performance and power consumption, we implement the microarchitectural units of the proposed architecture, including instruction security unit, data security unit at the Register Transfer Level (RTL). And we use Verilog Compiler Simulator to simulate our SecNVDLA project. We also use Synopsys Design Compiler (L-2016.03-SP5) and a SMIC 65nm standard-cell library for the baseline to synthesize the proposed architecture and obtain the area and energy consumption of the hardware logic units.

We choose to implement SecNVDLA based on NVDLA with the small system model. NVDLA comes from open source community [18]. Table 2 summarizes the configurations of small system model. It contains 1024 MAC units and 256KB convolutional buffer. We add a 256KB global buffer for fused layer of DNN model, and apply verilog to implement AES-CTR encryption algorithm, RSA, LSRF and *bypass and link check*.

Table 2: The configurations of NVDLA with small system model.

Clock	INT16/FP16	INT8	Conv Buffer	DRAM BW
200Mhz	512 MACs	1024 MACs	256KB	10GB/s

Workloads. Several typical DNN networks are selected for performance and energy evaluation. Table 3 summarizes the evaluated

networks which contain the pertinent parameters such as released year, model size, number of layers and classification accuracy.

Table 3: Workloads, their released year, model size, number of layers and classification accuracy.

Network	Year	Model size (MB)	Layers	Classification accuracy
AlexNet	2012	224	8	72.6%
GoogLeNet	2015	54	58	84.4%
ResNet50	2016	102	50	80.2%

Energy measurements. For the SecNVDLA project, we use the synthesis results with SMIC 65nm standard-cell library and reported numbers.

Performance measurements. Three neural networks are used to evaluate the performance impact of instruction security unit, data security unit and fused layer unit. Furthermore, overall benefits on the SecNVDLA are evaluated.

6.2 Experimental Results

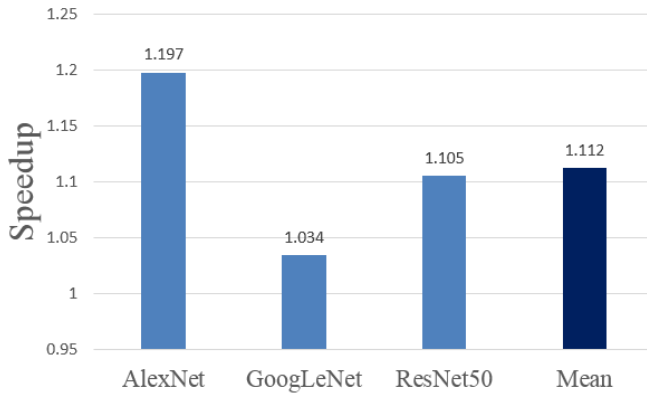
Performance impact of instruction security unit. In order to evaluate the effectiveness and feasibility of the instruction security unit, *bypass and link check* should be real-time to verify instructions. At the beginning, RSA computes the private key (256bit) for 32416 cycles. Compared to DNN accelerator without instruction security unit, performance drops by 1.037 $\times$, 1.024 $\times$ and 1.046 $\times$ for the AlexNet, GoogLeNet and ResNet50 networks, respectively. For AlexNet, GoogLeNet and ResNet50 networks, power consumption of instruction security unit is proportional to the number of instructions and model size.

Performance impact of data security unit. Table 4 shows the maximum performance loss for encrypting critical feature maps, compared with the original NVDLA architecture, it drops by 1.0345 $\times$, 1.2538 $\times$ and 1.0832 $\times$ for AlexNet, GoogLeNet and ResNet50 networks respectively. This is the most extreme case, the channel density is 0.5 and the channel correlation coefficient is 0. Using the three important criteria to select critical feature maps of AlexNet, GoogLeNet and ResNet50, we find the critical feature maps which satisfy both security and performance. Table 4 also

Table 4: The maximum performance loss of encrypting critical feature maps (Max Loss). Performance loss of encrypting critical feature maps (Normal Loss) and energy reduction.

Network	Max Loss	Normal Loss	Energy Reduction
AlexNet	1.0345×	1.0221×	1.4234×
GoogLeNet	1.2538×	1.1353×	2.3424×
ResNet50	1.0832×	1.0711×	3.2101×
Mean	1.1238×	1.0762×	2.3253×

shows the performance impact of encrypting critical feature maps compared with the original NVDLA architecture and also illustrates the energy reduction compared with encrypting all feature maps. Among all the DNN models, ResNet50 enjoys the highest energy reduction (3.2101×) and has the lowest average channel density (0.31). GoogLeNet has the highest performance loss with the lowest average channel correlation coefficient (0.14). It reduces the power consumption (2.3424×) while ensures the security of data security unit.

**Figure 4: The performance of the secure architecture of DNN accelerator**

Overall benefits with the secure architecture. Figure 4 illustrates the speedup with the secure architecture compared with the original architecture without secure countermeasures. In this setting, on average, it delivers 1.112× speedup and 1.096× energy increase, respectively. The minimum range of speedup is observed in GoogLeNet, in which the minimum speedup is 1.034×

Table 5 summarizes the area of the microarchitectural components in NVDLA and SecNVDLA. Overall, the secure accelerator needs 1.064× more area compared to NVDLA without the secure module. This increase in the area is mainly attributed to the added the security unit of DNN accelerator and on-chip buffer. The power consumption of the SecNVDLA increases 1.096× compared with NVDLA without secure countermeasures.

Compared with the original architecture without secure countermeasures, the proposed secure architecture of DNN accelerator enhances the security and improves performance by approximately 1.112×. These results endorse the feasibility and effectiveness of the *NPUFort*.

Table 5: The design area and power consumption of secure architecture on NVDLA

Accelerator	Area (um ²)	Power (mW)	Combinational Cell	Sequential Cell
NVDLA	4288329	625.8964	842138	65878
SecNVDLA	4563272	685.2384	865079	68865
Overhead	6.4%	9.6%	3.1%	4.5%

7 RELATED WORK

Recently, there have been some attempts of model inversion attack to obtain the structure and weights of DNN. Attacker usually adopts side-channel information [11, 16, 34] to retrieve the parameters of neural network. Batina et al.[2] presented an approach which recovers the input image from the collected power traces without knowing the detailed parameters of the neural network. They implemented the attack on an FPGA-based DNN implementation by observing power side-channel.

Although we represent the concrete study on the general DNN accelerator in the context of reverse engineering DNN models, there are still many problems worthy of study such as hiding information leaks through memory access patterns, memory isolation and similar SGX defense mechanism [7]. Certainly, general DNN accelerator also has many other secure problems. For example, global register and general register are easily designed as a hardware trojan in NVDLA. Utilizing clock gating may also lead to information leakage.

Some recent studies [3, 14, 24, 25, 28] have introduced lower overhead and high-performance ORAM algorithms by relaxing the threat model to hide write or read access patterns, but they also increase the impact of memory wall and decrease the performance of the DNN accelerator. We also try to design lower overhead memory traffic obfuscation by adding cryptographic logic in the memory to satisfy the performance and security requirements of the DNN accelerator [27]. The general DNN accelerators shown in this paper do not cover some emerging accelerators. The proposed security architecture can be applied to these DNN accelerators which contain a sparse architecture that uses pruning technology.

8 CONCLUSION

In this paper, we presented a secure architecture of DNN accelerator called *NPUFort* against model inversion attack, which contains an instruction security unit, a data security unit. Instruction security unit and data security unit which protect instruction and data from being accessed by attacker result in approximately 1.118× performance loss. Data security unit also obscures the boundary between layers and reduces timing side channel information leakage. These experiments demonstrate the security and effectiveness of the proposed secure architecture of DNN accelerator with improving performance by approximately 1.112×

ACKNOWLEDGEMENTS

This work was supported by National Science Foundation of China for Excellent Young Scholars under grant No.61522212, and the

Strategic Priority Research Program of Chinese Academy of Sciences under grant No. XD-C02000000. We wish to thank the anonymous reviewers for their valuable comments and suggestions.

REFERENCES

- [1] V Aklaghi, Amir Yazdanbakhsh, Kambiz Samadi, H Esmailzadeh, and RK Gupta. 2018. Snapea: Predictive early activation for reducing computation in deep convolutional neural networks. *ISCA*.
- [2] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2018. Csi neural network: Using side-channels to recover your artificial neural network information. *arXiv preprint arXiv:1810.09076* (2018).
- [3] Erik-Oliver Blass, Travis Mayberry, Guevara Noubir, and Kaan Onarlioglu. 2014. Toward robust hidden volumes using write-only oblivious RAM. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 203–214.
- [4] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. *ACM Sigplan Notices* 49, 4 (2014), 269–284.
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [6] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *ACM SIGARCH Computer Architecture News*, Vol. 44. IEEE Press, 367–379.
- [7] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. *IACR Cryptology ePrint Archive* 2016, 086 (2016), 1–118.
- [8] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A Horowitz, and William J Dally. 2016. EIE: efficient inference engine on compressed deep neural network. In *Computer Architecture (ISCA), 2016 ACM/IEEE 43rd Annual International Symposium on*. IEEE, 243–254.
- [9] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149* (2015).
- [10] Hengyuan Hu, Rui Peng, Yu-Wing Tai, and Chi-Keung Tang. 2016. Network trimming: A data-driven neuron pruning approach towards efficient deep architectures. *arXiv preprint arXiv:1607.03250* (2016).
- [11] Weizhe Hua, Zhiru Zhang, and G Edward Suh. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *Proceedings of the 55th Annual Design Automation Conference*. ACM, 4.
- [12] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Computer Architecture (ISCA), 2017 ACM/IEEE 44th Annual International Symposium on*. IEEE, 1–12.
- [13] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2016. Pruning filters for efficient convnets. *arXiv preprint arXiv:1608.08710* (2016).
- [14] Lichun Li and Anwitaman Datta. 2017. Write-only oblivious RAM-based privacy-preserved access of outsourced data. *International Journal of Information Security* 16, 1 (2017), 23–42.
- [15] Shaoli Liu, Zidong Du, Jinhua Tao, Dong Han, Tao Luo, Yuan Xie, Yunji Chen, and Tianshi Chen. 2016. Cambricon: An instruction set architecture for neural networks. In *ACM SIGARCH Computer Architecture News*, Vol. 44. IEEE Press, 393–405.
- [16] Hoda Naghibijouybari, Ajaya Neupane, Zhiyun Qian, and Nael Abu-Ghazaleh. 2018. Rendered Insecure: GPU Side Channel Attacks are Practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2139–2153.
- [17] NVIDIA. Hardware Architectural Specification. 2018. <http://nvidia.org/hw/v1/hwarch.html>
- [18] NVIDIA. NVDLA HW. 2018. <https://github.com/nvidia/hw>
- [19] NVIDIA. NVDLA Primer. 2018. <http://nvidia.org/primer.html>
- [20] NVIDIA. Unit Description. 2018. http://nvidia.org/hw/v1/ias/unit_description.html#tab-sdp-supported-use-scenarios
- [21] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. 2017. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, 506–519.
- [22] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Pugliese, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. Scnn: An accelerator for compressed-sparse convolutional neural networks. In *ACM SIGARCH Computer Architecture News*, Vol. 45. ACM, 27–40.
- [23] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. 2016. DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks. In *USENIX Security Symposium*. 565–581.
- [24] Joydeep Rakshit and Kartik Mohanram. 2018. LEO: Low Overhead Encryption ORAM for Non-Volatile Memories. *IEEE Computer Architecture Letters* 17, 2 (2018), 100–104.
- [25] Daniel S Roche, Adam Aviv, Seung Geol Choi, and Travis Mayberry. 2017. Deterministic, stash-free write-only oram. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 507–521.
- [26] Sara Sabour, Yanshuai Cao, Fartash Faghri, and David J Fleet. 2015. Adversarial manipulation of deep representations. *arXiv preprint arXiv:1511.05122* (2015).
- [27] Marc Sánchez-Artigas. 2018. Enhancing Tree-Based ORAM Using Batched Request Reordering. *IEEE Transactions on Information Forensics and Security* 13, 3 (2018), 590–604.
- [28] Emil Stefanov, Marten Van Dijk, Elaine Shi, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. 2013. Path ORAM: an extremely simple oblivious RAM protocol. In *Proceedings of the 2013 ACM SIGSAC conference on Computer and communications security*. ACM, 299–310.
- [29] Pedro Tabacof, Julia Tavares, and Eduardo Valle. 2016. Adversarial images for variational autoencoders. *arXiv preprint arXiv:1612.00155* (2016).
- [30] Shruti Tople, Karan Grover, Shweta Shinde, Ranjita Bhagwan, and Ramachandran Ramjee. 2018. Privado: Practical and Secure DNN Inference. *arXiv preprint arXiv:1810.00602* (2018).
- [31] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs. In *USENIX Security Symposium*. 601–618.
- [32] Binghui Wang and Neil Zhenqiang Gong. 2018. Stealing hyperparameters in machine learning. *arXiv preprint arXiv:1802.05351* (2018).
- [33] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. 2015. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *Security and Privacy (SP), 2015 IEEE Symposium on*. IEEE, 640–656.
- [34] Mengjia Yan, Christopher Fletcher, and Josep Torrellas. 2018. Cache telepathy: Leveraging shared resource attacks to learn dnn architectures. *arXiv preprint arXiv:1808.04761* (2018).
- [35] Tien-Ju Yang, Yu-Hsin Chen, and Vivienne Sze. 2016. Designing energy-efficient convolutional neural networks using energy-aware pruning. *arXiv preprint arXiv:1611.05128* (2016).